DG INTERNS HUB · CYBERSECURITY INTERNSHIP

Week 6

Web Security & Vulnerability Testing — VAPT Basics

Studying the OWASP Top 10 and setting up a legal, self-hosted lab for web application testing.

Detail	Value
Name	Ayan Ansari
Intern ID	DG/SEPTEMBER/CYBER/014
Program	DG Interns Hub – Cybersecurity Internship
Assignment	Week 6 – Web Security & Vulnerability Testing (VAPT Basics)
Lab target	DVWA, installed and run on my own Kali Linux machine
Tools	Burp Suite, OWASP ZAP, DVWA
Date	October 2026

Contents

1. **Introduction to VAPT** 2
2. **Tools used** 3
3. **Lab environment setup** 4
4. **OWASP Top 10 concepts** 5
5. **Mitigation methods** 8

1. Introduction to VAPT

VAPT stands for Vulnerability Assessment and Penetration Testing. A vulnerability assessment looks at a system and lists its weak spots. A penetration test goes further and tries to prove that a weak spot can actually be used, always with permission from the owner.

Web application testing is one of the most common forms of VAPT, because websites are often the first thing exposed to the public internet. The OWASP Top 10 is the standard reference list for the most serious and most common risks found in web applications, and this week's task is built around four of those risks: SQL Injection, Cross-Site Scripting, Broken Authentication and Security Misconfiguration.

Why it matters

- Most real-world breaches exploit well-known, well-documented weaknesses rather than exotic techniques.
- A practice environment that is deliberately vulnerable lets a learner see these weaknesses safely, on a system nobody else depends on.
- Understanding how a flaw works is what makes the prevention advice meaningful, instead of just a checklist.

2. Tools used

Tool	What it is	Role in this assignment
Burp Suite	An intercepting proxy that sits between the browser and a website, recording every request and response.	Used to capture and inspect HTTP traffic to and from the local test target.
OWASP ZAP	A free, open-source web application scanner, maintained by OWASP itself.	Used to run automated checks against a web application and list what it finds.
DVWA (Damn Vulnerable Web Application)	A web app built on purpose with common vulnerabilities, used worldwide for learning.	Installed as a local, self-hosted practice target. It never leaves my own machine.

Rules followed

Every tool here was installed and run only on my own machine. DVWA was never exposed to the internet or any network beyond my own device, and no system outside my own control was scanned or tested.

3. Lab environment setup

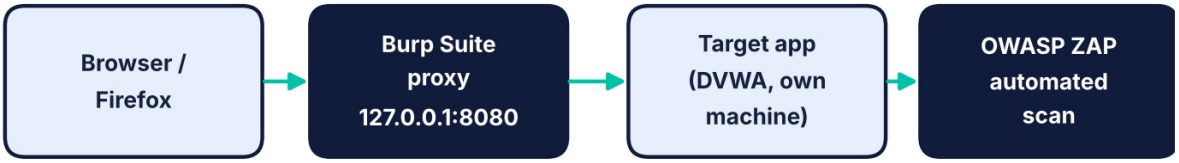


Figure 1 – How the tools fit together in the lab

Burp Suite

Burp Suite was opened with a temporary project and default settings. Its proxy listener was confirmed on 127.0.0.1:8080, which is the standard local address Burp uses to sit between a browser and a target application.

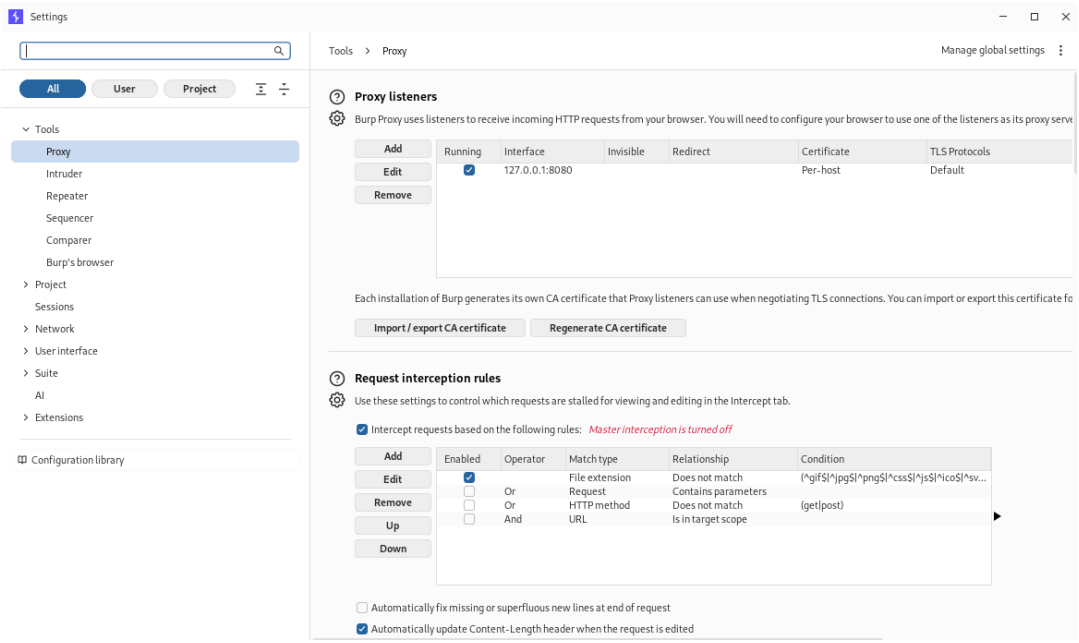


Figure 2 – Burp Suite proxy listener running on 127.0.0.1:8080

OWASP ZAP

OWASP ZAP (version 2.15.0) was installed and opened in Standard Mode. Its Quick Start screen offers an Automated Scan option, which is the entry point for running a scan against a target application.

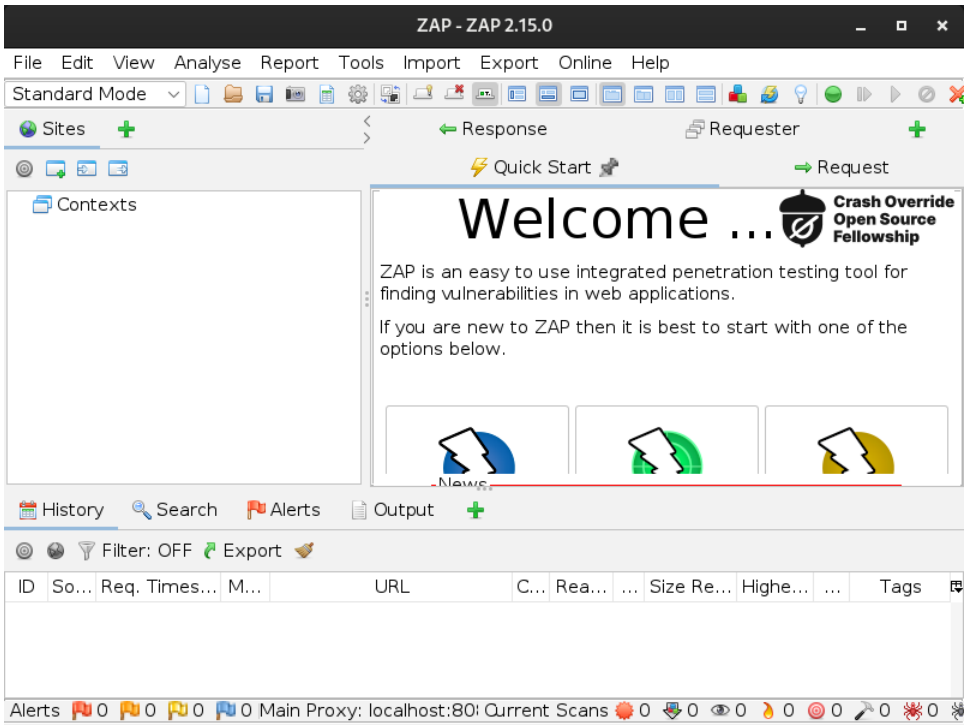


Figure 3 – OWASP ZAP 2.15.0 main window

DVWA

DVWA was installed on my Kali Linux machine using the project's own installer script. The installation downloaded the application from its official GitHub repository, set up the MariaDB database, configured the web server and PHP, and finished by initialising the database tables.

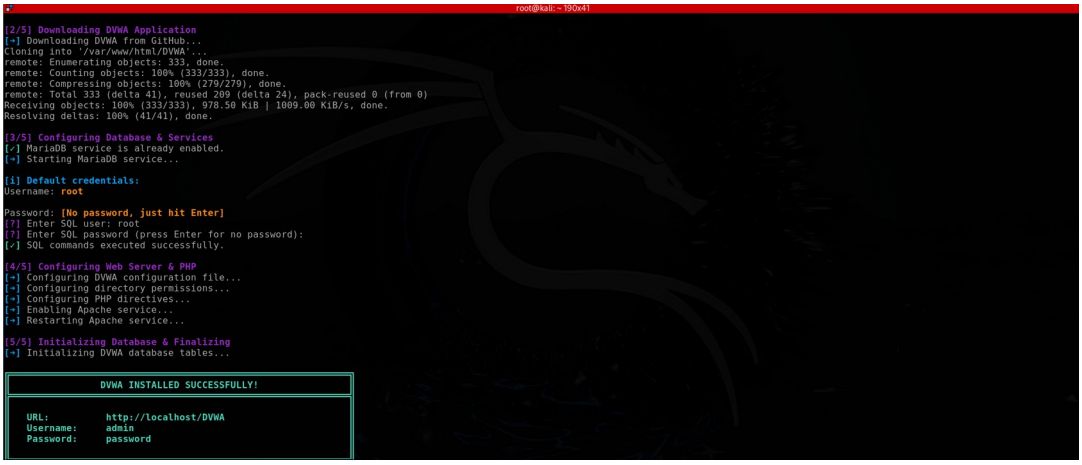


Figure 4 – DVWA installed successfully, with its local URL and default login shown

The installer reported the application reachable at http://localhost/DVWA with the default admin credentials, which only work on this local installation.

4. OWASP Top 10 concepts

These four topics were studied in depth as part of this assignment. A one-page summary of each is below; full notes with simple analogies and prevention checklists are included separately.

SQL Injection

Happens when user input is joined directly into a database query instead of being treated strictly as data. If the input contains database syntax, the database cannot tell it apart from the application's own instructions.

- Impact: unauthorised access to data, changed or deleted records, and in severe cases full database compromise.
- Prevention: parameterised queries (prepared statements), input validation, and least-privilege database accounts.

Cross-Site Scripting (XSS)

Happens when a web page displays untrusted input as if it were part of the page itself, so a browser runs it as code. It is split into reflected (bounced back in one response), stored (saved and shown to every visitor) and DOM-based (built by the page's own script).

- Impact: stolen session cookies, fake page content, and actions performed in the victim's name.
- Prevention: output encoding for the right context, a Content Security Policy, and HttpOnly/Secure cookies.

Broken Authentication

Covers weaknesses in how a system checks identity or manages a login session: no limit on login attempts, weak or default passwords, predictable session identifiers, or sessions that never expire.

- Impact: account takeover, and one weak account exposing many others through password reuse.
- Prevention: multi-factor authentication, login attempt limits, strong random session IDs, and salted password hashing (bcrypt/Argon2).

Security Misconfiguration

Happens when otherwise-safe software is deployed with unsafe settings: default accounts left active, unnecessary features switched on, detailed error messages shown to visitors, or security headers missing.

- Impact: information disclosure that helps an attacker plan further steps, or direct access through a default login.
- Prevention: a hardened baseline configuration, removing unused features, changing all defaults, and regular review after any change.

5. Mitigation methods

Vulnerability	Primary mitigation	Supporting measures
SQL Injection	Parameterised queries / prepared statements	Input validation, least-privilege DB accounts, generic error messages
XSS	Context-aware output encoding	Content Security Policy, HttpOnly + Secure cookies, safe templating
Broken Authentication	Multi-factor authentication	Login attempt limits, strong session IDs, salted password hashing
Security Misconfiguration	A hardened, consistent baseline configuration	Remove unused features, change defaults, security headers, regular patching

General principle

Across all four areas, the common thread is the same: never trust input, and never leave a default setting unchecked. These fixes are well documented, inexpensive to apply during development, and far cheaper than responding to a breach after the fact.

Closing note

This week built the foundation for hands-on web application testing: understanding the four vulnerability classes conceptually, and setting up Burp Suite, OWASP ZAP and DVWA as a safe, self-contained lab. Thank you to DG Interns Hub for structuring the learning this way.